



GESTIÓN DE BASES DE DATOS RELACIONALES MEDIANTE SQLITE EN WINDOWS

Inst: Instituto superior tecnológico

Araoz pinto

Esc.Pro: Desarrollo de sistemas de
Información

Curso: Sistemas operativos

Tema: SQLITE EN WINDOWS

Orcid: 0009-0000-9066-1889

Docente: Franklin Escobedo

Est: Patrick Negrete Mateo

Ciclo: Primer ciclo

Fecha: 15 de junio de 2026

Resumen

El presente informe academico tiene como objetivo documentar el proceso de diseno, implementacion y administracion de bases de datos relacionales utilizando el motor SQLite dentro del entorno operativo Microsoft Windows, correspondiente a la semana ocho del curso de Sistemas Operativos. El trabajo abarca tres ejes fundamentales: una investigacion sobre la arquitectura interna de SQLite, la resolucion practica del caso de la Ferreteria 'El Constructor' mediante la consola de comandos (CMD), y la respuesta a un cuestionario tecnico de veinte preguntas.

A traves de la simulacion practica en CMD se validaron operaciones transaccionales de creacion de tablas, restricciones de clave primaria, manipulacion de datos mediante sentencias DML y el uso de funciones de agregacion SQL. Los resultados demuestran que SQLite es una solucion sumamente eficiente para entornos de desarrollo rapido y sistemas embebidos, capacitando al estudiante en los principios de persistencia de datos bajo el estandar SQL.

Palabras clave: SQLite, base de datos relacional, CMD, DDL, DML, clave primaria, funciones de agregacion, inventario.

1. Introduccion

Dentro del campo del desarrollo de software y la administracion de sistemas informaticos, la persistencia de datos representa uno de los pilares mas criticos para el funcionamiento de cualquier aplicacion moderna. Los sistemas gestores de bases de datos relacionales (RDBMS) de nivel empresarial, como PostgreSQL, Oracle o Microsoft SQL Server, funcionan bajo un esquema clasico cliente-servidor que demanda infraestructura dedicada, procesos de fondo independientes y una carga administrativa considerable.

Como respuesta a esta complejidad, SQLite surgio como una alternativa ligera y eficaz para escenarios embebidos, dispositivos moviles y prototipado rapido. Su principal caracteristica radica en que incorpora un motor SQL completo, transaccional y totalmente autocontenido en una unica biblioteca escrita en lenguaje C. Esta practica guia al estudiante en la configuracion de variables de entorno en Windows, la creacion de bases de datos mediante el Simbolo del Sistema y el dominio de sentencias estructuradas DDL y DML.



Figura 2. Arquitectura interna de SQLite: capas principales del motor embebido.

Los objetivos de aprendizaje de esta semana incluyen comprender el diseno estructurado de registros, la aplicacion de restricciones logicas de integridad, y la automatizacion de consultas orientadas al sector industrial. Estos conocimientos sientan las bases para la comprension de sistemas de gestion de datos de mayor escala.

2. Metodologia

Para el desarrollo practico de la guia de laboratorio se aplico un enfoque experimental y empirico fundamentado en el uso de la interfaz de consola nativa de Windows. El entorno tecnologico estuvo conformado por una estacion de trabajo con Windows 11 y los binarios oficiales precompilados de SQLite3 (version de 32/64 bits para Windows).

2.1. Fase de Configuracion del Sistema

En primera instancia se realizo la descarga de los archivos sqlite-tools y sqlite-dll desde el sitio oficial de SQLite. Posteriormente, los binarios se descomprimieron en un directorio comun (por ejemplo C:\sqlite) y dicha ruta se incorporo a la variable de entorno PATH del sistema operativo, lo que permite invocar el comando sqlite3 de forma global desde cualquier ubicacion del Simbolo del Sistema, sin necesidad de navegar al directorio de instalacion cada vez.

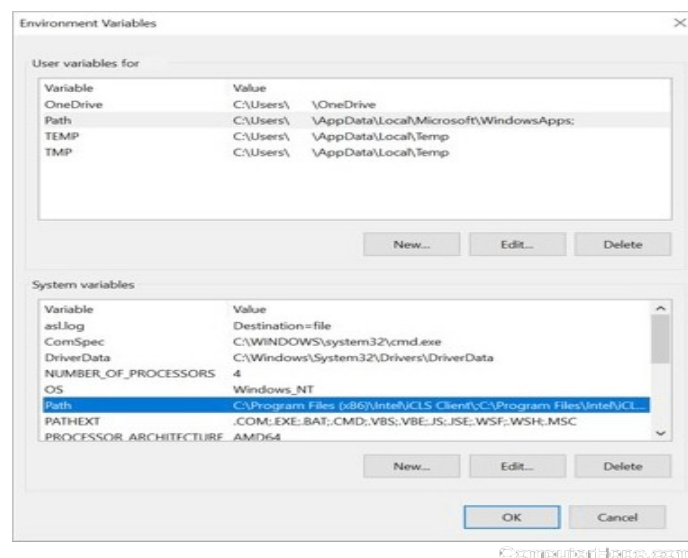


Figura 3. Configuracion de la variable de entorno PATH en Windows 11 para integrar SQLite.

2.2. Fase de Modelamiento - DDL

Una vez configurado el entorno, se procedio a crear el archivo fisico de la base de datos y a declarar las tablas correspondientes. Durante esta fase se asignaron tipos de datos adecuados (INTEGER, TEXT,

REAL) y se definieron restricciones de integridad como la clave primaria (PRIMARY KEY) y la restriccion de no nulo (NOT NULL), elementos fundamentales para garantizar la consistencia de los datos almacenados.

2.3. Fase de Operacion - DML

Con la estructura de datos definida, se ejecutaron inserciones en bloque, actualizaciones condicionales mediante la clausula WHERE y eliminaciones logicas y fisicas de registros. Cada operacion fue verificada inmediatamente mediante consultas SELECT para confirmar la correcta aplicacion de los cambios en la base de datos.

2.4. Fase de Consulta y Analisis Avanzado

Finalmente, se extrajeron registros de forma selectiva empleando filtros de patrones de texto con el operador LIKE, funciones de agregacion como SUM, AVG y COUNT, y ordenamientos ascendentes y descendentes mediante ORDER BY. Esta fase permite transformar datos brutos en informacion util para la toma de decisiones.

Figura 4. Uso de SQLite mediante la interfaz de linea de comandos de Windows (CMD).

3. Caso Practico: Ferreteria 'El Constructor'

El nucleo operativo de la practica consistio en desarrollar un sistema basico de control de inventarios para la empresa ferreteria 'El Constructor', con el proposito de gestionar de manera eficiente los articulos disponibles, sus existencias y costos unitarios. A continuacion se presenta la secuencia completa de comandos ejecutados en el entorno interactivo de SQLite.

3.1. Creacion de la Base de Datos e Inicializacion

Para comenzar, se invoca el ejecutable de SQLite pasando como argumento el nombre del archivo que se desea crear o abrir. Si el archivo no existe, SQLite lo genera automaticamente en el directorio activo:

```
-- Paso 1: Crear o abrir la base de datos de la ferreteria
C:\> sqlite3 ferreteria.db

SQLite version 3.45.0
Enter ".help" for usage hints.
sqlite>
```

3.2. Creacion de la Tabla de Inventario (DDL)

Se definio la tabla 'productos' con cuatro campos: un identificador entero que actua como clave primaria autoincremental, una descripcion textual, la cantidad en stock y el precio unitario en formato decimal. Las restricciones NOT NULL aseguran que no se puedan insertar registros con campos criticos vacios:

```
-- Paso 2: Crear la tabla de inventario con tipos de datos y restricciones
CREATE TABLE productos (
    codigo      INTEGER PRIMARY KEY,
    descripcion TEXT    NOT NULL,
    cantidad    INTEGER NOT NULL,
    precio      REAL    NOT NULL
);
```

3.3. Insercion de Registros Iniciales (DML - INSERT)

Se insertaron diez productos representativos de una ferreteria real, abarcando herramientas manuales, herramientas electricas y materiales de construccion, con sus respectivas cantidades en almacen y precios unitarios en soles:

```
-- Paso 3: Insertar 10 registros de materiales de construccion
INSERT INTO productos VALUES (1, 'Martillo de una 16oz', 25,
35.50);
INSERT INTO productos VALUES (2, 'Alicate de presion 10', 15,
42.00);
INSERT INTO productos VALUES (3, 'Destornillador estrella Set x6', 30,
28.90);
INSERT INTO productos VALUES (4, 'Cinta metrica 5m antideslizante', 50,
15.20);
INSERT INTO productos VALUES (5, 'Taladro percutor 600W', 8,
189.00);
INSERT INTO productos VALUES (6, 'Amoladora angular 4.5 pulgadas', 12,
145.00);
INSERT INTO productos VALUES (7, 'Caja de clavos 2 pulgadas x1kg', 100,
8.50);
INSERT INTO productos VALUES (8, 'Bolsa de cemento Portland tipo I', 200,
26.00);
INSERT INTO productos VALUES (9, 'Pala de punta redonda mango madera', 20,
48.00);
INSERT INTO productos VALUES (10, 'Carretilla de metal 5.5 pies cubicos', 10,
125.50);
```

3.4. Consulta, Actualizacion y Eliminacion de Datos

Se valido el contenido de la tabla, se actualizo la cantidad disponible del producto con codigo 3, y se elimino el producto con codigo 10 del inventario. Cada operacion fue verificada con una nueva consulta SELECT:

```
-- Paso 4: Mostrar todos los registros para validacion
```



```
SELECT * FROM productos;
```

```
-- Paso 5: Actualizar el stock del producto codigo 3 (de 30 a 45 unidades)
```

```
UPDATE productos SET cantidad = 45 WHERE codigo = 3;
```

```
-- Paso 6: Eliminar el producto con codigo 10 del inventario
```

```
DELETE FROM productos WHERE codigo = 10;
```

```
-- Paso 7: Verificar el estado final de la tabla
```

```
SELECT * FROM productos;
```

4. Desarrollo del Reto Final

El Reto Final requirio la resolucio de consultas analiticas avanzadas empleando funciones de agregacion y filtros condicionales estructurados. A continuacion se presenta la resolucio tecnica de cada una de las seis actividades planteadas, con su respectiva explicacion operativa:

Figura 5. Principales sentencias del Lenguaje de Manipulacion de Datos (DML) en SQL.

Actividad 1: Ordenamiento Alfabetico por Descripcion

Para presentar los registros de manera organizada alfabeticamente se utiliza la clausula ORDER BY sobre el campo de texto 'descripcion' con la palabra clave ASC (ascendente), lo que ordena los resultados de la A a la Z:

```
SELECT * FROM productos ORDER BY descripcion ASC;
```

Actividad 2: Ordenamiento por Cantidad de Mayor a Menor

La ordenacion inversa permite identificar de inmediato los articulos con mayor volumen en almacen. Para ello se emplea ORDER BY con el modificador DESC (descendente) aplicado al campo numerico 'cantidad':

```
SELECT * FROM productos ORDER BY cantidad DESC;
```

Actividad 3: Búsqueda por Patron de Texto con LIKE

El operador LIKE, combinado con el comodin '%', permite filtrar registros cuya descripción comience por una letra específica. En este caso, se recuperan los productos cuyo nombre inicia con la letra 'C' (Cinta, Caja de clavos, Cemento, Carretilla):

```
SELECT * FROM productos WHERE descripcion LIKE 'C%';
```

Actividad 4: Conteo Total de Registros

La función de agregación COUNT(*) devuelve un valor entero que representa el número total de filas almacenadas en la tabla. El alias 'total_productos' hace que la columna de resultado sea más legible:

```
SELECT COUNT(*) AS total_productos FROM productos;
```

Actividad 5: Calculo del Precio Promedio

La función AVG calcula el promedio aritmético de todos los valores en la columna numérica 'precio', dividiendo la suma total entre el número de registros existentes. El resultado refleja el costo medio del inventario:

```
SELECT AVG(precio) AS precio_promedio FROM productos;
```

Actividad 6: Cierre Seguro de la Sesión SQLite

Para finalizar la sesión interactiva de SQLite de manera segura se utiliza el comando de punto '.exit', que cierra el descriptor del archivo físico de la base de datos y retorna el control al Símbolo del Sistema de Windows sin riesgo de corrupción de datos:

```
.exit
```

5. Cuestionario Tecnico Avanzado

A continuacion se presentan las veinte preguntas del cuestionario tecnico, respondidas con lenguaje tecnico adecuado para el nivel universitario:

1. ¿Que es SQLite?

SQLite es un sistema de gestion de bases de datos relacionales (RDBMS) de codigo abierto, autocontenido, sin servidor y sin necesidad de configuracion previa. Su principal caracteristica es que almacena toda la estructura de datos, tablas, indices y registros dentro de un unico archivo de disco ordinario, sin requerir ningun proceso externo en segundo plano para funcionar.

2. ¿Que ventajas ofrece SQLite frente a otros motores de bases de datos?

Las principales ventajas de SQLite son su ligereza extrema (minimo consumo de memoria RAM), portabilidad total al ser un solo archivo movil entre plataformas, ausencia total de administracion de servidores, cumplimiento completo de las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) para las transacciones, y velocidad sobresaliente en operaciones de lectura local. Esto lo convierte en la base de datos mas implementada a nivel mundial, presente en todos los dispositivos moviles Android e iOS, navegadores web y sistemas embebidos.

3. ¿Que comando permite abrir SQLite desde CMD?

El acceso a SQLite desde el Simbolo del Sistema de Windows se realiza ejecutando el comando 'sqlite3' seguido, de forma opcional, por el nombre del archivo de base de datos que se desea crear o gestionar. Si el archivo aun no existe, SQLite lo crea automaticamente en el directorio activo. Ejemplo: sqlite3 ferreteria.db

4. ¿Que diferencia existe entre SQLite y SQL Server?

La diferencia fundamental radica en su arquitectura. SQLite es un motor embebido en el que la biblioteca accede directamente al archivo fisico de datos, siendo ideal para aplicaciones locales, moviles y de escritorio. Microsoft SQL Server, en cambio, es un sistema cliente-servidor de nivel empresarial que requiere un servicio de sistema operativo en ejecucion permanente, gestion de red, configuracion de puertos y recursos de hardware robustos. Mientras SQLite es gratuito y de dominio publico, SQL Server tiene costo de licenciamiento comercial.

5. ¿Que es una tabla?

En el modelo relacional, una tabla es una estructura logica bidimensional organizada en filas (registros o tuplas) y columnas (atributos o campos). Cada tabla representa una entidad del modelo de datos (por ejemplo, 'productos', 'clientes') y los datos son almacenados bajo un esquema formal previamente definido mediante sentencias DDL.

6. ¿Que es una clave primaria (PRIMARY KEY)?

La clave primaria es una restriccion de integridad de entidad aplicada a una columna o conjunto de columnas cuyo objetivo es identificar de forma univoca e irrepetible cada fila de una tabla. SQLite garantiza que los valores en la columna PRIMARY KEY nunca se dupliquen y nunca sean nulos, siendo el principal mecanismo de identificacion de registros dentro del modelo relacional.

7. ¿Que funcion cumple la sentencia INSERT?

La sentencia INSERT INTO es la instruccion DML encargada de agregar nuevas filas con datos especificos dentro de una tabla existente. Su sintaxis basica permite insertar valores para todos los campos en el orden de definicion de la tabla, o especificar de forma explicita los nombres de las columnas a poblar, lo que otorga mayor flexibilidad cuando se tienen campos opcionales.

8. ¿Que funcion cumple la sentencia SELECT?

La sentencia SELECT es la instruccion fundamental del Lenguaje de Consultas Estructurado (SQL) y permite recuperar, filtrar, ordenar y proyectar datos almacenados en una o multiples tablas. Mediante clausulas adicionales como WHERE, ORDER BY, GROUP BY y HAVING, se puede refinar la extraccion de informacion con un alto nivel de precision y control.

9. ¿Que funcion cumple la clausula WHERE?

La clausula WHERE actua como un filtro logico condicional que limita las filas procesadas por una sentencia SELECT, UPDATE o DELETE, incluyendo unicamente aquellas que satisfacen una condicion booleana determinada. Admite operadores de comparacion (<, >, =, !=), operadores logicos (AND, OR, NOT) y operadores especiales como LIKE, IN y BETWEEN.

10. ¿Que funcion cumple ORDER BY?

La clausula ORDER BY se utiliza para establecer el criterio de ordenamiento del conjunto de resultados devuelto por una consulta SELECT. Se puede aplicar de forma ascendente (ASC, valor predeterminado) o descendente (DESC) sobre una o varias columnas, permitiendo presentar la informacion en el orden mas util para el usuario o sistema receptor.

11. Sentencia para crear una tabla llamada empleados.

```
CREATE TABLE empleados (id INTEGER PRIMARY KEY, nombre TEXT NOT NULL, puesto TEXT, salario REAL);
```

12. Sentencia para insertar un registro en la tabla empleados.

```
INSERT INTO empleados VALUES (1, 'Carlos Mendoza', 'Administrador', 2500.00);
```

13. Sentencia para mostrar todos los registros de la tabla empleados.

```
SELECT * FROM empleados;
```

14. Sentencia para actualizar el salario de un empleado.

```
UPDATE empleados SET salario = 2800.00 WHERE id = 1;
```

15. Sentencia para eliminar un registro de la tabla empleados.

```
DELETE FROM empleados WHERE id = 1;
```

16. ¿Como contar registros en una tabla?

Para obtener el numero total de filas en una tabla se emplea la funcion de agregacion COUNT junto con SELECT. La expresion COUNT(*) cuenta todas las filas independientemente de valores nulos, mientras que COUNT(columna) excluye los nulos de la columna especificada. Ejemplo: SELECT COUNT(*) AS total FROM empleados;

17. ¿Como calcular el promedio de una columna?

El promedio aritmetico de una columna numerica se obtiene mediante la funcion de agregacion AVG, que divide la suma total de los valores no nulos entre el numero de registros considerados. Ejemplo aplicado: SELECT AVG(salario) AS salario_promedio FROM empleados;

18. ¿Como buscar registros que comiencen con una letra especifica?

Se combina la clausula WHERE con el operador LIKE y el comodin '%' para filtrar cadenas de texto por patron. El comodin '%' representa cualquier secuencia de caracteres, mientras que '_' representa exactamente un caracter. Para buscar nombres que inicien con 'A': SELECT * FROM empleados WHERE nombre LIKE 'A%';

19. ¿Como ordenar registros de mayor a menor?

Para ordenar los resultados de una consulta en forma descendente se agrega al final la instruccion ORDER BY seguida del nombre de la columna y la palabra clave DESC. Ejemplo practico para empleados por salario de mayor a menor: SELECT * FROM empleados ORDER BY salario DESC;

20. ¿Como salir de SQLite de forma segura?

Para cerrar la sesion interactiva de SQLite y retornar al sistema operativo se utilizan los comandos de terminal propios del cliente: `.exit` o `.quit` (siempre con el punto antepuesto). Estos comandos aseguran que el archivo de base de datos quede correctamente guardado y cerrado antes de devolver el control al Simbolo del Sistema.

6. Discussion

El analisis de los resultados obtenidos durante la practica confirma que la arquitectura sin servidor de SQLite le confiere propiedades de ligereza excepcionales para el almacenamiento relacional de pequena y mediana escala. Al prescindir de sockets de red y de procesos de sistema operativo en escucha permanente, las latencias para transacciones individuales se reducen al minimo, quedando limitadas practicamente por la velocidad de lectura y escritura del disco local (SSD o HDD).

No obstante, es importante reconocer las limitaciones inherentes a este motor. SQLite implementa un mecanismo de bloqueo a nivel de archivo completo cuando se ejecuta una transaccion de escritura (INSERT, UPDATE, DELETE), lo cual implica que todo el archivo queda inaccesible para operaciones de escritura concurrentes de otros procesos o usuarios durante ese periodo. Esta restriccion lo hace inapropiado para sistemas web de alta concurrencia, donde cientos o miles de usuarios podrian intentar modificar datos de manera simultanea.

En contraste, para entornos academicos, aplicaciones de escritorio, sistemas embebidos, aplicaciones moviles y casos de uso como el inventario de la Ferreteria 'El Constructor', SQLite representa la solucion optica por su simplicidad operativa, portabilidad y cumplimiento del estandar SQL. La tabla comparativa siguiente resume las diferencias clave entre SQLite y sistemas RDBMS de mayor escala:

Criterio	SQLite	SQL Server
Arquitectura	Embebida / sin servidor	Cliente-Servidor
Instalacion	Sin instalacion requerida	Instalacion y configuracion complejas
Almacenamiento	Un unico archivo .db	Archivos del sistema + servicios Windows
Concurrencia	Limitada (bloqueo por archivo)	Alta concurrencia multiusuario
Administracion	Ninguna	DBA dedicado recomendado
Uso tipico	Apps moviles, desktop, IoT	Sistemas corporativos, web de alta escala
Licencia	Dominio publico	Comercial (de pago)

Otro aspecto relevante identificado durante la practica es la importancia critica de configurar correctamente las variables de entorno del sistema operativo. Esta configuracion, aunque parezca un paso tecnico menor,

es la que permite la abstraccion de la herramienta del sistema de archivos, habilitando su invocacion desde cualquier ubicacion del sistema sin requerir rutas absolutas en cada llamada.

7. Conclusiones

1. La configuracion correcta de las variables de entorno PATH en Windows es un paso de infraestructura critico que permite abstraer herramientas externas del sistema de archivos, facilitando su invocacion global desde la linea de comandos sin necesidad de rutas absolutas en cada llamada.
2. SQLite demuestra ser un motor relacional excepcionalmente agil y funcional para entornos academicos y aplicaciones reales embebidas, garantizando el cumplimiento de la sintaxis estandar SQL sin la sobrecarga computacional que implica administrar servidores de bases de datos tradicionales.
3. El dominio de las sentencias DDL y DML, asi como de las funciones de agregacion COUNT, AVG y SUM, resulta indispensable para transformar registros de datos brutos en informacion util, estructurada y accionable para la toma de decisiones empresariales.
4. Las capacidades analiticas ejercitadas a traves de ordenamientos complejos y filtros basados en expresiones condicionales validan que el control de inventarios de pequenas empresas puede ser automatizado de manera rigurosa mediante estructuras relacionales ligeras como SQLite.
5. Aunque SQLite es idoneo para un amplio rango de aplicaciones locales, sus limitaciones de concurrencia (bloqueo por archivo) hacen necesaria la migracion hacia arquitecturas cliente-servidor como PostgreSQL o SQL Server cuando el sistema escale a multiples usuarios concurrentes o requiera alta disponibilidad en red.

8. Referencias Bibliograficas

Las siguientes fuentes fueron consultadas y citadas en el presente informe, siguiendo el formato de la 7ma edicion de las normas APA:

Hipp, D. R. (2020). SQLite: A self-contained, serverless, zero-configuration, transactional SQL database engine. SQLite Official Documentation. <https://www.sqlite.org/>

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Fundamentos de bases de datos (7ma ed.). McGraw-Hill Education.

Date, C. J. (2004). Introduccion a los sistemas de bases de datos (8va ed.). Pearson Prentice Hall.

Owens, M. (2006). The definitive guide to SQLite. Apress.

Microsoft Corporation. (2023). Environment variables (Windows). Microsoft Documentation. <https://docs.microsoft.com/en-us/windows/win32/procthread/environment-variables>